Lecture 1: Introduction

Patterns, methods, traits and generics

Willem Vanhulle

DevLab Rust 2025

Tuesday November 4, 2025

github.com/wvhulle/rust-course-ghent

What is the meaning of the word ‘Rust’ in this context?

What is the meaning of the word ‘Rust’ in this context?

It’s the name of a kind of organism: robust ‘funghi’

(Don’t listen to your Flemish grandma, it is **not chilling**, it’s “Roest” in Dutch.)

1.1. Participants

Round of introductions:

- What is your name?
- What would you like to learn?

1.1. Participants

Round of introductions:

- What is your name?
- What would you like to learn?
  - Microcontrollers?
  - Web servers?
  - Robotics?

1.1. Participants

Round of introductions:

- What is your name?
- What would you like to learn?
  - Microcontrollers?
  - Web servers?
  - Robotics?
- What is your experience with programming?
- What is your experience with Rust?

1.2. Me

1.2.1. Experience

Teaching experience:

- Mathematics tutor for 5 years (students 10-25 years old): analysis, statics, algebra, calculus, …
- Part-time mathematics teacher at adult school for 1 year
- Organising internal Rust workshops at software companies (2 years)
- Organising systems programming workshops at SysGhent.be (1 year)

1.2. Me

1.2.1. Experience

Teaching experience:

- Mathematics tutor for 5 years (students 10-25 years old): analysis, statics, algebra, calculus, ...
- Part-time mathematics teacher at adult school for 1 year
- Organising internal Rust workshops at software companies (2 years)
- Organising systems programming workshops at SysGhent.be (1 year)

Rust experience:

- Control small-scale fermentation robot and peripherals with Rust (1 year)
- Architect and implement remote control system for freight trains in Rust (1 year)
- Various hobby projects and conference talk

1.2. Me

1.2.1. Experience

Teaching experience:

- Mathematics tutor for 5 years (students 10-25 years old): analysis, statics, algebra, calculus, ...
- Part-time mathematics teacher at adult school for 1 year
- Organising internal Rust workshops at software companies (2 years)
- Organising systems programming workshops at SysGhent.be (1 year)

Rust experience:

- Control small-scale fermentation robot and peripherals with Rust (1 year)
- Architect and implement remote control system for freight trains in Rust (1 year)
- Various hobby projects and conference talk

Contact information:

- Questions: willemvanhulle@protonmail.com
- Website: willemvanhulle.tech, github.com/wvhulle

1.2. Me

1.2.2. Hobby projects

Latest Rust hobby projects: nu-lint (linter NuShell) and a `firewall-manager`

Info

Nushell is a modern shell written in Rust.

Makes pipelines easier with modern syntax and types:

```
date now                    # 1: today
| $in + 1day                # 2: tomorrow
| format date '%F'          # 3: Format as YYYY-MM-DD
| $'($in) Report'           # 4: Format the directory name
| mkdir $in                 # 5: Create the directory
```

(playground link)

1.2. Me

1.2.2. Hobby projects

Latest Rust hobby projects: `nu-lint` (linter NuShell) and a `firewall-manager`

Info

Nushell is a modern shell written in Rust.

Makes pipelines easier with modern syntax and types:

```
date now                    # 1: today
| $in + 1day                # 2: tomorrow
| format date '%F'          # 3: Format as YYYY-MM-DD
| $'($in) Report'           # 4: Format the directory name
| mkdir $in                 # 5: Create the directory
```

(playground link)

Free Nu workshop

On Wed, 3rd of December, I will give a free workshop on NuShell in this location. Register at SysGent meetup page.

1.3. Misconceptions about Rust

It’s too difficult.

1.3. Misconceptions about Rust

It's too difficult.

No, it is only hard in the first 1 month.
After that, you will experience relief. You will be able to sleep at night (no more segfaults!).

It's not used in industry.

1.3. Misconceptions about Rust

It's too difficult.

No, it is only hard in the first 1 month.
After that, you will experience relief. You will be able to sleep at night (no more segfaults!).

It's not used in industry.

No, big companies are actually using it: Google, Amazon, Microsoft.
Near Ghent: Robovision, Barco, OTIV, … . In coming years more and more: Keysight, Infrabel, …

Rust is only for systems programming.

1.3. Misconceptions about Rust

It's too difficult.

No, it is only hard in the first 1 month.
After that, you will experience relief. You will be able to sleep at night (no more segfaults!).

It's not used in industry.

No, big companies are actually using it: Google, Amazon, Microsoft.
Near Ghent: Robovision, Barco, OTIV, … . In coming years more and more: Keysight, Infrabel, …

Rust is only for systems programming.

No, you can build:

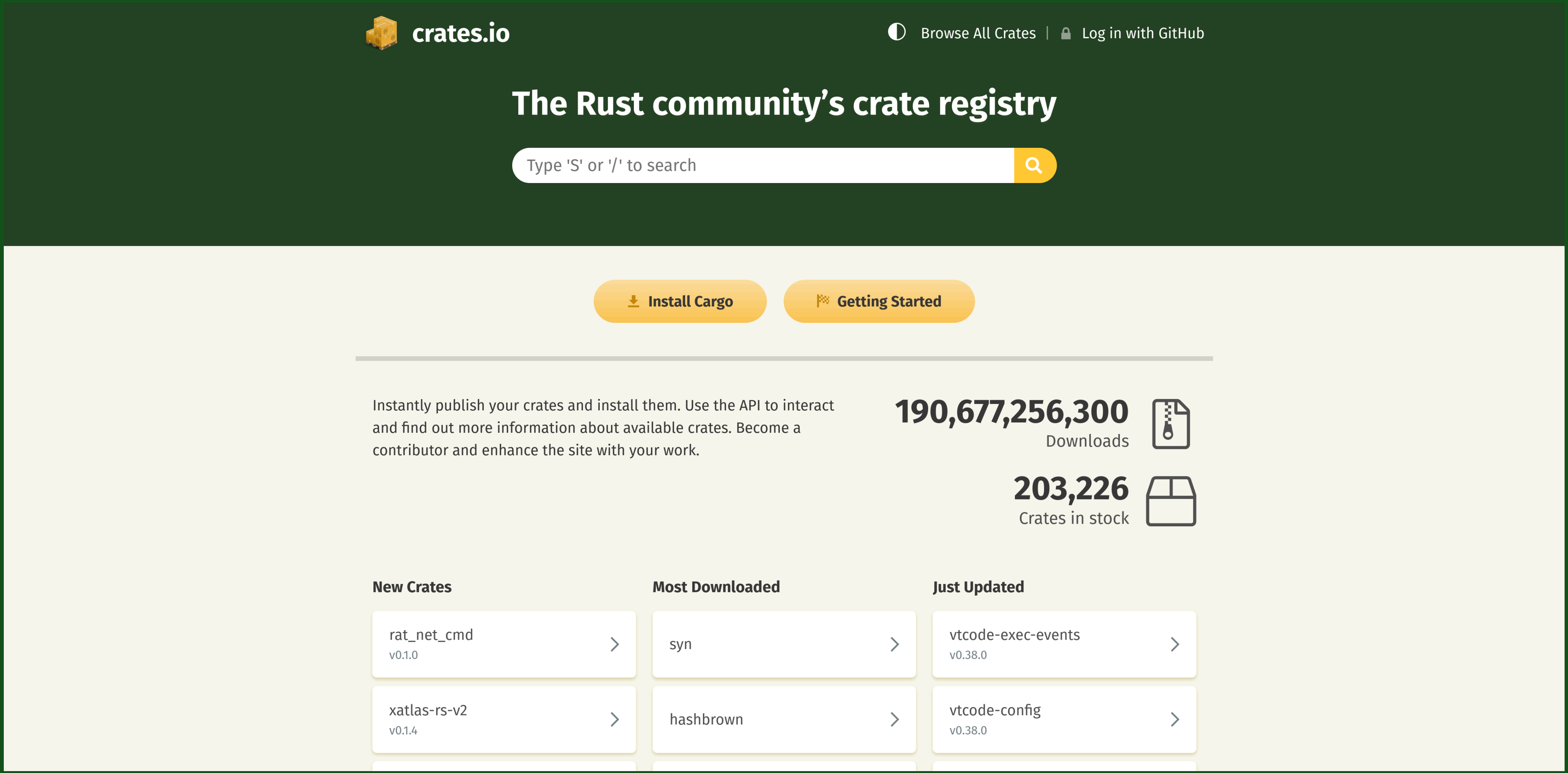
- Web applications (backend and frontend)
- Command-line tools
- Data science and machine learning

1.4. Ultimate goal

By the end of this course, you should be able to **publish your own code** to the Rust package registry crates.io.

Warning

Start thinking now what you would like to build and let me know before next session!


crates.io
Browse All Crates
Log in with GitHub
The Rust community's crate registry
Type 'S' or '/' to search
Install Cargo
Getting Started
Instantly publish your crates and install them. Use the API to interact and find out more information about available crates. Become a contributor and enhance the site with your work.
190,677,256,300
Downloads
203,226
Crates in stock
New Crates
Most Downloaded
Just Updated
rat_net_cmd
v0.1.0
syn
vtcode-exec-events
v0.38.0
xatlas-rs-v2
v0.1.4
hashbrown
vtcode-config
v0.38.0

1.5. Learning material

Theory:

- These **slides contain extra diagrams**:
  - https://github.com/wvhulle/rust-course-ghent
  - written in Typst (a Rust based typesetting language like LaTeX)
- Book: “Programming Rust” by Jim Blandy, any edition ($\sim$ 35€, ask me for e-book)

1.5. Learning material

Theory:

- These **slides contain extra diagrams**:
  - https://github.com/wvhulle/rust-course-ghent
  - written in Typst (a Rust based typesetting language like LaTeX)
- Book: “Programming Rust” by Jim Blandy, any edition ($\sim$ 35€, ask me for e-book)

Exercises:

- Exercise from Google’s Rust course
- More exercises https://exercism.org/tracks/rust
- Lots of official documentation https://doc.rust-lang.org/std/index.html

Our aim in this course: **$>$ 50% exercises!**

std
1.91.0
(f8297e351 2025-10-28)

Crate std

Since 1.0.0 · Source

Search Settings Help Summary

The Rust Standard Library

The Rust Standard Library is the foundation of portable Rust software, a set of minimal and battle-tested shared abstractions for the broader Rust ecosystem. It offers core types, like `Vec<T>` and `Option<T>`, library-defined operations on language primitives, standard macros, I/O and multithreading, among many other things.

`std` is available to all Rust crates by default. Therefore, the standard library can be accessed in `use` statements through the path `std`, as in `use std::env`.

How to read this documentation

If you already know the name of what you are looking for, the fastest way to find it is to use the search button at the top of the page.

Otherwise, you may want to jump to one of these useful sections:

- `std::*` modules
- Primitive types
- Standard macros
- The Rust Prelude

If this is your first time, the documentation for the standard library is written to be casually perused. Clicking on interesting things

1.6. Setup for exercises

You don't have to install Rust for this first lecture:

- Solve during lectures in playground https://play.rust-lang.org
- Code samples have a "playground link" at the bottom right

1.6. Setup for exercises

You don't have to install Rust for this first lecture:

- Solve during lectures in playground https://play.rust-lang.org
- Code samples have a "playground link" at the bottom right

Do you prefer to do exercises locally?

- Find copies of exercises in the `session-N/examples/` folders
- Run them with `cargo run --example s1e1-fibonacci` (from root or session folder).

Warning

Turn off AI extensions for exercises in Rust! Don't use AI to write code for you.

1.6. Setup for exercises

You don't have to install Rust for this first lecture:

- Solve during lectures in playground https://play.rust-lang.org
- Code samples have a "playground link" at the bottom right

Do you prefer to do exercises locally?

- Find copies of exercises in the `session-N/examples/` folders
- Run them with `cargo run --example s1e1-fibonacci` (from root or session folder).

Warning

Turn off AI extensions for exercises in Rust! Don't use AI to write code for you.

Still need harder challenges:

- Find additional exercises on https://exercism.org/tracks/rust/exercises.
- Work on your own project!

Back to Exercise

Rust / Space Age

src/lib.rs | Cargo.toml

```rust
// The code below is a stub. Just enough to satisfy the compiler.
// In order to pass the tests you can add-to or change any of this code.

#[derive(Debug)]
pub struct Duration;

impl From<u64> for Duration {
    fn from(s: u64) -> Self {
        todo!("s, measured in seconds: {s}")
    }
}

pub trait Planet {
    fn years_during(d: &Duration) -> f64 {
        todo!("convert a duration ({d:?}) to the number of years on this planet for that duration");
    }
}
```

Stuck? Get help | Run Tests | Submit

Instructions | **Tests** | Results | ChatGPT

```rust
use space_age::*;

fn assert_in_delta(expected: f64, actual: f64) {
    let diff: f64 = (expected - actual).abs();
    let delta: f64 = 0.01;
    if diff > delta {
        panic!("Your result of {actual} should be within {delta} of the expected result {expected}")
    }
}

#[test]
fn age_on_earth() {
    let seconds = 1_000_000_000;
    let duration = Duration::from(seconds);
    let output = Earth::years_during(&duration);
    let expected = 31.69;
    assert_in_delta(expected, output);
}
```

1.7. Course contents

Each session is 2 hours long. Longer break in the middle. Half theory, half exercises.

1. Introduction, **rust basics, patterns, methods, traits, generics** (+ homework)
2. Functional programming & standard library
3. Ownership & borrowing fundamentals
4. Smart pointers & memory management
5. Iterators, modules & testing
6. Error handling & concurrency
7. Asynchronous programming

1.8. Rust today

Part 1: Rust fundamentals:

- Types and values
- Control flow basics
- Tuples and arrays
- References
- User-defined types

1.8. Rust today

Part 1: Rust fundamentals:

- Types and values
- Control flow basics
- Tuples and arrays
- References
- User-defined types

Part 2: Important Rust concepts:

- Pattern matching
- Methods
- Traits (maybe at home)
- Generics (maybe at home)

2.1. Hello, world!

```rust
fn main() {
    println!("Hello, world!");
}
```

(playground link)

2.1. Hello, world!


```rust
fn main() {
    println!("Hello, world!");
}
```

(playground link)

- `fn` defines a function
- Statements end with `;`
- `main` is the entry point
- Rust strings are UTF-8 encoded and can contain any Unicode character.

2.1. Hello, world!


```rust
fn main() {
    println!("Hello, world!");
}
```

(playground link)

- `fn` defines a function
- Statements end with `;`
- `main` is the entry point
- Rust strings are UTF-8 encoded and can contain any Unicode character.

What does the `!` in `println!` signify?

2.1. Hello, world!

```rust
fn main() {
    println!("Hello, world!");
}
```

(playground link)

- `fn` defines a function
- Statements end with `;`
- `main` is the entry point
- Rust strings are UTF-8 encoded and can contain any Unicode character.

What does the `!` in `println!` signify?

It's a macro, not a function. Macros expand at compile time and can take variable arguments.

2.1. Hello, world!

```rust
fn main() {
    println!("Hello, world!");
}
```

(playground link)

- `fn` defines a function
- Statements end with `;`
- `main` is the entry point
- Rust strings are UTF-8 encoded and can contain any Unicode character.

What does the `!` in `println!` signify?

It's a macro, not a function. Macros expand at compile time and can take variable arguments.

Can you call `println!` without any arguments?

2.1. Hello, world!

```rust
fn main() {
    println!("Hello, world!");
}
```

(playground link)

- `fn` defines a function
- Statements end with `;`
- `main` is the entry point
- Rust strings are UTF-8 encoded and can contain any Unicode character.

What does the `!` in `println!` signify?

It's a macro, not a function. Macros expand at compile time and can take variable arguments.

Can you call `println!` without any arguments?

No, it requires at least a format string: `println!("")` or use `println!()` in newer Rust versions.

2.2. Variables

Type safety with static typing:

```rust
fn main() {
  let x: i32 = 10;
  println!("x: {x}");
  // x = 20;
  // println!("x: {x}");
}
```

(playground link)

2.2. Variables

Type safety with static typing:

```rust
fn main() {
  let x: i32 = 10;
  println!("x: {x}");
  // x = 20;
  // println!("x: {x}");
}
```

(playground link)

What happens if we uncomment the two lines?

2.2. Variables

Type safety with static typing:

```rust
fn main() {
    let x: i32 = 10;
    println!("x: {x}");
    // x = 20;
    // println!("x: {x}");
}
```

(playground link)

What happens if we uncomment the two lines?

Variables are immutable by default. Use `mut` to make them mutable.

2.2. Variables

Type safety with static typing:

```rust
fn main() {
  let x: i32 = 10;
  println!("x: {x}");
  // x = 20;
  // println!("x: {x}");
}
```

(playground link)

What happens if we uncomment the two lines?

Variables are immutable by default. Use `mut` to make them mutable.

Are type annotations required?

2.2. Variables

Type safety with static typing:

```rust
fn main() {
    let x: i32 = 10;
    println!("x: {x}");
    // x = 20;
    // println!("x: {x}");
}
```

(playground link)

What happens if we uncomment the two lines?

Variables are immutable by default. Use `mut` to make them mutable.

Are type annotations required?

Not required for local variables (if the compiler **can infer**).

2.3. Values

Basic built-in types:

- Signed integers: `i8`, `i16`, `i32`, `i64`, `i128`, `isize`
- Unsigned integers: `u8`, `u16`, `u32`, `u64`, `u128`, `usize`

2.3. Values

Basic built-in types:

- Signed integers: `i8`, `i16`, `i32`, `i64`, `i128`, `isize`
- Unsigned integers: `u8`, `u16`, `u32`, `u64`, `u128`, `usize`

When should you use `isize` and `usize`?

2.3. Values

Basic built-in types:

- Signed integers: `i8`, `i16`, `i32`, `i64`, `i128`, `isize`
- Unsigned integers: `u8`, `u16`, `u32`, `u64`, `u128`, `usize`

When should you use `isize` and `usize`?

Use `usize` for pointers and array indices.

2.3. Values

Basic built-in types:

- Signed integers: `i8`, `i16`, `i32`, `i64`, `i128`, `isize`
- Unsigned integers: `u8`, `u16`, `u32`, `u64`, `u128`, `usize`

When should you use `isize` and `usize`?

Use `usize` for pointers and array indices.

- Floating point: `f32`, `f64`
- Boolean: `bool` (`true` or `false`)

2.3. Values

Basic built-in types:

- Signed integers: `i8`, `i16`, `i32`, `i64`, `i128`, `isize`
- Unsigned integers: `u8`, `u16`, `u32`, `u64`, `u128`, `usize`

When should you use `isize` and `usize`?

Use `usize` for pointers and array indices.

- Floating point: `f32`, `f64`
- Boolean: `bool` (`true` or `false`)

2.3. Values

Does this compile?

```rust
fn main() {
    let x: i32 = 5;
    let y: i64 = x;
}
```

(playground link)

Can you compare values of different numeric types directly (e.g., `i32` and `i64`)?

2.3. Values

Does this compile?

```rust
fn main() {
    let x: i32 = 5;
    let y: i64 = x;
}
```

(playground link)

Can you compare values of different numeric types directly (e.g., `i32` and `i64`)?

No! Rust has no implicit type conversions. You must use explicit casts with `as`: `x as i64 == y`

2.3. Values

Does this compile?

```rust
fn main() {
    let x: i32 = 5;
    let y: i64 = x;
}
```

(playground link)

Can you compare values of different numeric types directly (e.g., `i32` and `i64`)?

No! Rust has no implicit type conversions. You must use explicit casts with `as`: `x as i64 == y`

- Character: `char`, written `'x'` (Unicode scalar value, 4 bytes)

Warning

Every cast that may lose precision must be explicit with `as`!

2.4. Arithmetic

```rust
fn interproduct(a: i32, b: i32, c: i32) -> i32 {
  return a * b + b * c + c * a;
}

fn main() {
    println!("result: {}", interproduct(120, 100, 248));
}
```

(playground link)

2.4. Arithmetic

```rust
fn interproduct(a: i32, b: i32, c: i32) -> i32 {
  return a * b + b * c + c * a;
}

fn main() {
    println!("result: {}", interproduct(120, 100, 248));
}
```

(playground link)

What happens when integer overflow occurs?

2.4. Arithmetic

```rust
fn interproduct(a: i32, b: i32, c: i32) -> i32 {
  return a * b + b * c + c * a;
}

fn main() {
    println!("result: {}", interproduct(120, 100, 248));
}
```

(playground link)

What happens when integer overflow occurs?

It is defined behavior in release mode (wrap around), but panics in debug mode.

2.5. Question

```rust
fn main() {
    let mut x = 4;
    --x;
    print!("{}{}", --x, --x);
}
```

(playground link)

What is the output of this program?

2.5. Question

```rust
fn main() {
    let mut x = 4;
    --x;
    print!("{}{}", --x, --x);
}
```

(playground link)

What is the output of this program?

Short answer: 4. Longer answer: Rust does not have a unary increment or decrement operator.

2.6. Type Inference


```rust
fn takes_u32(x: u32) {
  println!("u32: {x}");
}

fn takes_i8(y: i8) {
    println!("i8: {y}");
}

fn main() {
    let x = 10;
    let y = 20;

    takes_u32(x);
    takes_i8(y);
    // takes_u32(y);
}
```

(playground link)

2.6. Type Inference

```
fn takes_u32(x: u32) {
  println!("u32: {x}");
}

fn takes_i8(y: i8) {
    println!("i8: {y}");
}

fn main() {
    let x = 10;
    let y = 20;

    takes_u32(x);
    takes_i8(y);
    // takes_u32(y);
}
```

(playground link)

Does this code compile?

2.6. Type Inference

```
fn takes_u32(x: u32) {
  println!("u32: {x}");
}

fn takes_i8(y: i8) {
    println!("i8: {y}");
}

fn main() {
    let x = 10;
    let y = 20;

    takes_u32(x);
    takes_i8(y);
    // takes_u32(y);
}
```

(playground link)

Does this code compile?

Yes, Rust infers `x` as `u32` and `y` as `i8`.

2.6. Type Inference


```
fn takes_u32(x: u32) {
  println!("u32: {x}");
}

fn takes_i8(y: i8) {
    println!("i8: {y}");
}

fn main() {
    let x = 10;
    let y = 20;

    takes_u32(x);
    takes_i8(y);
    // takes_u32(y);
}
```

(playground link)

Does this code compile?

Yes, Rust infers `x` as `u32` and `y` as `i8`.

What happens if we uncomment the last line?

2.6. Type Inference

```rust
fn takes_u32(x: u32) {
  println!("u32: {x}");
}

fn takes_i8(y: i8) {
    println!("i8: {y}");
}

fn main() {
    let x = 10;
    let y = 20;

    takes_u32(x);
    takes_i8(y);
    // takes_u32(y);
}
```

(playground link)

Does this code compile?

Yes, Rust infers `x` as `u32` and `y` as `i8`.

What happens if we uncomment the last line?

It does not compile: type mismatch.

2.6. Type Inference

```rust
fn takes_u32(x: u32) {
  println!("u32: {x}");
}

fn takes_i8(y: i8) {
    println!("i8: {y}");
}

fn main() {
    let x = 10;
    let y = 20;

    takes_u32(x);
    takes_i8(y);
    // takes_u32(y);
}
```

(playground link)

Does this code compile?

Yes, Rust infers `x` as `u32` and `y` as `i8`.

What happens if we uncomment the last line?

It does not compile: type mismatch.

What is the default integer type in Rust?

2.6. Type Inference


```rust
fn takes_u32(x: u32) {
  println!("u32: {x}");
}

fn takes_i8(y: i8) {
    println!("i8: {y}");
}

fn main() {
    let x = 10;
    let y = 20;

    takes_u32(x);
    takes_i8(y);
    // takes_u32(y);
}
```

(playground link)

Does this code compile?

Yes, Rust infers `x` as `u32` and `y` as `i8`.

What happens if we uncomment the last line?

It does not compile: type mismatch.

What is the default integer type in Rust?

It is `i32` unless otherwise specified.

2.6. Type Inference

Who was Pingala?

2.6. Type Inference

Who was Pingala?

An ancient Indian scholar (circa 200 BC) who described the Fibonacci sequence in his work on prosody.

Info

The Fibonacci sequence begins with [0, 1]. For n > 1, the next number is the sum of the previous two.

Complete the code in `session1/examples/s1e1-fib.rs`.

2.6. Type Inference

2.6.1. Follow-up questions

When will this function panic?

2.6. Type Inference

2.6.1. Follow-up questions

When will this function panic?

- When `n` is too large and the result overflows `u32`.
- When recursion depth exceeds the stack size (around 10,000).

2.6.1. Follow-up questions

When will this function panic?

- When `n` is too large and the result overflows `u32`.
- When recursion depth exceeds the stack size (around 10,000).

How to fix these issues?

2.6. Type Inference

2.6.1. Follow-up questions

When will this function panic?

- When `n` is too large and the result overflows `u32`.
- When recursion depth exceeds the stack size (around 10,000).

How to fix these issues?

- Make it iterative.
- Use `u128` or external `bigint` crate for large Fibonacci numbers.

3.1. Blocks and scopes

```rust
fn main() {
    let z = 13;
    let x = {
        let y = 10;
        dbg!(y);
        z - y
    };
    dbg!(x);
    // dbg!(y);
}
```

(playground link)

3.1. Blocks and scopes

```rust
fn main() {
    let z = 13;
    let x = {
        let y = 10;
        dbg!(y);
        z - y
    };
    dbg!(x);
    // dbg!(y);
}
```

(playground link)

What is the output of this program?

3.1. Blocks and scopes

```rust
fn main() {
    let z = 13;
    let x = {
        let y = 10;
        dbg!(y);
        z - y
    };
    dbg!(x);
    // dbg!(y);
}
```

(playground link)

What is the output of this program?

The output is `x = 3`.

3.1. Blocks and scopes


```rust
fn main() {
    let z = 13;
    let x = {
        let y = 10;
        dbg!(y);
        z - y
    };
    dbg!(x);
    // dbg!(y);
}
```

(playground link)

What is the output of this program?

The output is `x = 3`.

What happens if we uncomment the last line?

3.1. Blocks and scopes


```rust
fn main() {
    let z = 13;
    let x = {
        let y = 10;
        dbg!(y);
        z - y
    };
    dbg!(x);
    // dbg!(y);
}
```

(playground link)

What is the output of this program?

The output is `x = 3`.

What happens if we uncomment the last line?

Would not compile because `y` is out of scope: deallocated.

3.1. Blocks and scopes

```rust
fn main() {
    let z = 13;
    let x = {
        let y = 10;
        dbg!(y);
        z - y
    };
    dbg!(x);
    // dbg!(y);
}
```

(playground link)

What is the output of this program?

The output is `x = 3`.

What happens if we uncomment the last line?

Would not compile because y is out of scope: de-allocated.

Info

Rust is an **expression-based** language: almost everything is an expression that returns a value.

3.2. If expressions

```rust
fn main() {
    let x = 10;
    if x == 0 {
        println!("zero!");
    } else if x < 100 {
        println!("biggish");
    } else {
        println!("huge");
    }
}
```

(playground link)

3.2. If expressions

```rust
fn main() {
  let x = 10;
  let size = if x < 20 { "small" } else { "large" };
  println!("number size: {}", size);
}
```

(playground link)

What happens when we put a semicolon (`;`) behind the `"small"` string?

3.2. If expressions

```rust
fn main() {
  let x = 10;
  let size = if x < 20 { "small" } else { "large" };
  println!("number size: {}", size);
}
```

(playground link)

What happens when we put a semicolon (`;`) behind the `"small"` string?

It does not compile.

Warning

Semi-colons at the end of block **suppress block return values**.

3.3. Match expressions

```rust
fn main() {
    let val = 1;
    match val {
        1 => println!("one"),
        10 => println!("ten"),
        100 => println!("one hundred"),
        _ => {
            println!("something else");
        }
    }
}
```

(playground link)

3.3. Match expressions

```rust
fn main() {
    let val = 1;
    match val {
        1 => println!("one"),
        10 => println!("ten"),
        100 => println!("one hundred"),
        _ => {
            println!("something else");
        }
    }
}
```

(playground link)

What happens without the _ case?

3.3. Match expressions

```rust
fn main() {
  let val = 1;
  match val {
        1 => println!("one"),
        10 => println!("ten"),
        100 => println!("one hundred"),
        _ => {
            println!("something else");
        }
  }
}
```

(playground link)

What happens without the _ case?

It does not compile: non-exhaustive patterns.

Warning

All possible cases must be handled in a `match` expression (exhaustiveness is required).

3.3. Match expressions

```rust
fn main() {
  let flag = true;
  let val = match flag {
      true => 1,
      false => 0,
  };
  println!("The value of {flag} is {val}");
}
```

(playground link)

3.3. Match expressions

```rust
fn main() {
  let flag = true;
  let val = match flag {
      true => 1,
      false => 0,
  };
  println!("The value of {flag} is {val}");
}
```

(playground link)

Info

Every `match` is also an expression that returns a value.

3.3. Match expressions

```rust
fn main() {
  let flag = true;
  let val = match flag {
      true => 1,
      false => 0,
  };
  println!("The value of {flag} is {val}");
}
```

(playground link)

Info

Every `match` is also an expression that returns a value.

Warning

- All blocks are expressions! Arms may be blocks.
- Do not put semi-colons at the end of arms if you want to return a value!

3.4. Loops

```rust
fn main() {
    let mut x = 200;
    while x >= 10 {
        x = x / 2;
    }
    dbg!(x);
}
```

(playground link)

Info

`break`, `continue` and `return` work as expected. `loop` is `while true`.

3.4. Loops

```rust
fn main() {
    for x in 1..5 {
        dbg!(x);
    }

    for elem in [2, 4, 8, 16, 32] {
        dbg!(elem);
    }
}
```

(playground link)

3.4. Loops

```rust
fn main() {
    for x in 1..5 {
        dbg!(x);
    }

    for elem in [2, 4, 8, 16, 32] {
        dbg!(elem);
    }
}
```

(playground link)

What can we write inside the `in` of a `for` loop?

3.4. Loops

```rust
fn main() {
    for x in 1..5 {
        dbg!(x);
    }

    for elem in [2, 4, 8, 16, 32] {
        dbg!(elem);
    }
}
```

(playground link)

What can we write inside the `in` of a `for` loop?

Any iterable type. Implemented with `IntoIterator` trait (see later).

3.5. Functions

```rust
fn gcd(a: u32, b: u32) -> u32 {
    if b > 0 { gcd(b, a % b) } else { a }
}

fn main() {
    dbg!(gcd(143, 52));
}
```

(playground link)

What is the return type of `main`?

3.5. Functions

```rust
fn gcd(a: u32, b: u32) -> u32 {
    if b > 0 { gcd(b, a % b) } else { a }
}

fn main() {
    dbg!(gcd(143, 52));
}
```

(playground link)

What is the return type of `main`?

It is `()`, the **unit type** (like `void` in C/C++) and optional.

3.5. Functions

```rust
fn gcd(a: u32, b: u32) -> u32 {
    if b > 0 { gcd(b, a % b) } else { a }
}

fn main() {
    dbg!(gcd(143, 52));
}
```

(playground link)

What is the return type of `main`?

It is `()`, the **unit type** (like `void` in C/C++) and optional.

How would you achieve function overloading in Rust?

3.5. Functions

```rust
fn gcd(a: u32, b: u32) -> u32 {
    if b > 0 { gcd(b, a % b) } else { a }
}

fn main() {
    dbg!(gcd(143, 52));
}
```

(playground link)

What is the return type of `main`?

It is `()`, the **unit type** (like `void` in C/C++) and optional.

How would you achieve function overloading in Rust?

Rust doesn't support traditional overloading. Instead, use: different names, generic functions with trait bounds, or method overloading through traits like `Add` or `From`.

3.6. Macros

- `println!(format)` prints to standard output with formatting.
- `dbg!(expr)` prints the value of `expr` to standard error for debugging.
- `todo!()` panics with a "not yet implemented" message.

3.6. Macros

- `println!(format)` prints to standard output with formatting.
- `dbg!(expr)` prints the value of `expr` to standard error for debugging.
- `todo!()` panics with a "not yet implemented" message.

Which macros should you use in tests?

3.6. Macros

- `println!(format)` prints to standard output with formatting.
- `dbg!(expr)` prints the value of `expr` to standard error for debugging.
- `todo!()` panics with a "not yet implemented" message.

Which macros should you use in tests?

- `assert!(condition)` to check if a condition is true.
- `assert_eq!(a, b)` to check if two values are equal.

```rust
fn factorial(n: u32) -> u32 {
    let mut product = 1;
    for i in 1..=n {
        product *= dbg!(i);
    }
    product
}
fn main() {
    let n = 4;
    println!("{n}! = {}", factorial(n));
}
```

(playground link)

3.7. Exercise: Collatz sequence

Solve the exercise about Collatz sequences by completing the code in `session1/examples/s1e2-collatz.rs`.

4.1. Arrays

Info

Arrays are fixed-size, **fixed-length contiguous collections** located on the stack

4.1. Arrays

Info

Arrays are fixed-size, **fixed-length contiguous collections** located on the stack

```rust
fn main() {
    let mut a: [i8; 5] = [5, 4, 3, 2, 1];
    a[2] = 0;
    println!("a: {a:?}");
}
```

(playground link)

4.1. Arrays

Info

Arrays are fixed-size, **fixed-length contiguous collections** located on the stack

```rust
fn main() {
    let mut a: [i8; 5] = [5, 4, 3, 2, 1];
    a[2] = 0;
    println!("a: {a:?}");
}
```

(playground link)

Can you omit the length of the array in the type annotation?

Info

Arrays are fixed-size, **fixed-length contiguous collections** located on the stack

```rust
fn main() {
    let mut a: [i8; 5] = [5, 4, 3, 2, 1];
    a[2] = 0;
    println!("a: {a:?}");
}
```

(playground link)

Can you omit the length of the array in the type annotation?

Yes, the compiler can **infer it from the initializer**. In other cases, it is required.

4.1. Arrays

```rust
fn main() {
    let mut a: [i8; 5] = [5, 4, 3, 2, 1];
    a[6] = 0;
    println!("a: {a:?}");
}
```

(playground link)

What happens when we run this program?

4.1. Arrays

```rust
fn main() {
    let mut a: [i8; 5] = [5, 4, 3, 2, 1];
    a[6] = 0;
    println!("a: {a:?}");
}
```

(playground link)

What happens when we run this program?

It panics at runtime: index out of bounds. Rust performs **bounds checking** on array accesses at runtime.

4.1. Arrays

```rust
fn main() {
    let mut a: [i8; 5] = [5, 4, 3, 2, 1];
    a[6] = 0;
    println!("a: {a:?}");
}
```

(playground link)

What happens when we run this program?

It panics at runtime: index out of bounds. Rust performs **bounds checking** on array accesses at runtime.

Warning

If possible, the compiler omits the bound checks. They may also be disabled explicitly by user.

4.1. Arrays

Have a look at this code:

```rust
fn get_index() -> usize { 6 }
fn main() {
    let mut a: [i8; 5] = [5, 4, 3, 2, 1];
    a[get_index()] = 0;
    println!("a: {a:?}");
}
```

(playground link)

Will the compiler implement run-time bounds checks?

4.1. Arrays

Have a look at this code:

```
fn get_index() -> usize { 6 }
fn main() {
    let mut a: [i8; 5] = [5, 4, 3, 2, 1];
    a[get_index()] = 0;
    println!("a: {a:?}");
}
```

(playground link)

Will the compiler implement run-time bounds checks?

Yes, because the index is not known at compile time.

4.1. Arrays

Have a look at this code:

```rust
fn get_index() -> usize { 6 }
fn main() {
    let mut a: [i8; 5] = [5, 4, 3, 2, 1];
    a[get_index()] = 0;
    println!("a: {a:?}");
}
```

(playground link)

Will the compiler implement run-time bounds checks?

Yes, because the index is not known at compile time.

Info

You could write `const fn get_index() -> usize { 6 }` to make it a compile-time constant and remove the runtime bounds check.

4.1. Arrays

Info

All arrays have a fixed size known at compile time.

4.1. Arrays

Info

All arrays have a fixed size known at compile time.

Warning

All arrays are stored on the stack by default.

4.1. Arrays

Info

All arrays have a fixed size known at compile time.

Warning

All arrays are stored on the stack by default.

Can the length of an array be modified at runtime?

4.1. Arrays

Info

All arrays have a fixed size known at compile time.

Warning

All arrays are stored on the stack by default.

Can the length of an array be modified at runtime?

No, but you can have arrays with mutable elements.

```rust
let mut a: [i8; 5] = [5, 4, 3, 2, 1];
a[2] = 0;
```

(playground link)

4.2. Debug output

Printing arrays for debugging:

```rust
fn main() {
    let a: [i8; 5] = [5, 4, 3, 2, 1];
    println!("a: {:?}", a);
}
```

(playground link)

4.2. Debug output

Printing arrays for debugging:

```rust
fn main() {
    let a: [i8; 5] = [5, 4, 3, 2, 1];
    println!("a: {:?}", a);
}
```

(playground link)

What does the `:?` format specifier do?

4.2. Debug output

Printing arrays for debugging:

```rust
fn main() {
    let a: [i8; 5] = [5, 4, 3, 2, 1];
    println!("a: {:?}", a);
}
```

(playground link)

What does the `:?` format specifier do?

It uses the `Debug` trait to format the value for debugging purposes. (Also used by `dbg!` and `eprintln!` macros.)

4.3. Tuples

Grouping values (of different types) without field names:

```rust
fn main() {
    let t: (i8, bool) = (7, true);
    dbg!(t.0);
    dbg!(t.1);
}
```

(playground link)

4.3. Tuples

Grouping values (of different types) without field names:

```rust
fn main() {
    let t: (i8, bool) = (7, true);
    dbg!(t.0);
    dbg!(t.1);
}
```

(playground link)

Tuples are limited

- You cannot use them in `for` loops.
- You cannot change their length.
- You cannot access elements by name.
- You cannot use array indexing syntax (e.g., `t[0]`).

4.4. Array iteration

Arrays are iterable.

```rust
fn main() {
    let primes = [2, 3, 5, 7, 11, 13, 17, 19];
    for prime in primes {
        for i in 2..prime {
            assert_ne!(prime % i, 0);
        }
    }
}
```

(playground link)

4.5. Patterns and destructuring

You can destructure tuples and arrays with **destructuring patterns**:

```rust
fn check_order(tuple: (i32, i32, i32)) -> bool {
    let (left, middle, right) = tuple;
    left < middle && middle < right
}

fn main() {
    let tuple = (1, 5, 3);
    println!(
        "{tuple:?}: {}",
        if check_order(tuple) { "ordered" } else { "unordered" }
    );
}
```

(playground link)

Info

An irrefutable destructuring pattern is a pattern that **always matches**.

4.6. Exercise: nested arrays

Implement a matrix transpose function that works on 3x3 matrices:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \to \begin{pmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{pmatrix}$$

```
fn transpose(matrix: [[i32; 3]; 3]) -> [[i32; 3]; 3] {
    todo!()
}
```

(playground link)

Find the exercise here: `session1/examples/s1e3-transpose.rs`.

5.1. Shared references

$\boxed{\text{Reference } \&T} \longrightarrow$

5.1. Shared references

Reference &T ⟶ **Referrent** T

```rust
fn main() {
    let a = 'A';
    let b = 'B';
    let mut r: &char = &a;
    dbg!(r);
}
```

(playground link)

5.1. Shared references

Reference &T ⟶ **Referrent** T

```rust
fn main() {
    let a = 'A';
    let b = 'B';
    let mut r: &char = &a;
    dbg!(r);
}
```

(playground link)

How is r displayed in the output?

5.1. Shared references

Reference &T ⟶ **Referrent** T

```rust
fn main() {
    let a = 'A';
    let b = 'B';
    let mut r: &char = &a;
    dbg!(r);
}
```

(playground link)

How is r displayed in the output?

It shows the value pointed to: 'A'.

5.1. Shared references

Reference &T ⟶ **Referent** T

```rust
fn main() {
    let a = 'A';
    let b = 'B';
    let mut r: &char = &a;
    dbg!(r);
}
```

(playground link)

How is r displayed in the output?

It shows the value pointed to: 'A'.

How can we print the reference itself (the memory address)?

5.1. Shared references

Reference `&T` ⟶ **Referent** `T`

```rust
fn main() {
    let a = 'A';
    let b = 'B';
    let mut r: &char = &a;
    dbg!(r);
}
```

(playground link)

How is `r` displayed in the output?

It shows the value pointed to: `'A'`.

How can we print the reference itself (the memory address)?

Use `println!("{r:p}")` or convert to raw pointer: `println!("{:p}", r as *const _)`

5.1. Shared references

Warning

Objects are shared immutably among different & through shared references.

```rust
fn main() {
    let mut c = 'C';
    let r: &char = &c;
    // *r = 'D';
    dbg!(r);
}
```

(playground link)

5.1. Shared references

Warning

Objects are shared immutably among different & through shared references.

```rust
fn main() {
    let mut c = 'C';
    let r: &char = &c;
    // *r = 'D';
    dbg!(r);
}
```

(playground link)

What happens if we uncomment the line that tries to modify `*r`?

5.1. Shared references

Warning

Objects are shared immutably among different & through shared references.

```rust
fn main() {
    let mut c = 'C';
    let r: &char = &c;
    // *r = 'D';
    dbg!(r);
}
```

(playground link)

What happens if we uncomment the line that tries to modify *r?

Objects cannot be modified through & references.

5.2. Dereferencing

References can be dereferenced automatically in certain cases.

In C++, you would dereference like this:

```cpp
char c = 'A';
char* r = &c;
std::cout << *r << std::endl;
```

(playground link)

In Rust, you can just write:

```rust
let c: char = 'A';
let r: &char = &c;
println!("{r}");
```

(playground link)

5.3. Difficult question

"What is the output of this Rust program?"

```rust
fn main() {
    let a;
    let a = a = true;
    print!("{}", std::mem::size_of_val(&a));
}
```

(playground link)

5.3. Difficult question

"What is the output of this Rust program?"

```
fn main() {
    let a;
    let a = a = true;
    print!("{}", std::mem::size_of_val(&a));
}
```

(playground link)

The variable `a` is shadowed by a new variable `a` which could be renamed `b`:

```
let a;
let b = a = true;
print!("{}", mem::size_of_val(&b));
```

(playground link)

5.3. Difficult question

```rust
let a;
let b = a = true;
print!("{}", mem::size_of_val(&b));
```

(playground link)

What is the output of this Rust program?

5.3. Difficult question

```rust
let a;
let b = a = true;
print!("{}", mem::size_of_val(&b));
```

(playground link)

What is the output of this Rust program?

The value of an assignment is `()`.

We can rewrite the program as:

```rust
let a = true;
let b = ();
print!("{}", mem::size_of_val(&b));
```

(playground link)

The `size_of_val` gives the size of `()` (the referent).

What is the size of `()`?

5.3. Difficult question

```rust
let a;
let b = a = true;
print!("{}", mem::size_of_val(&b));
```

(playground link)

What is the output of this Rust program?

The value of an assignment is `()`.

We can rewrite the program as:

```rust
let a = true;
let b = ();
print!("{}", mem::size_of_val(&b));
```

(playground link)

The `size_of_val` gives the size of `()` (the referent).

What is the size of `()`?

0 bytes.

5.4. Pointers

Every reference can be converted to a raw pointer, using a cast with `as`:

```
let x: i32 = 10;
let r: *const i32 = &x as *const i32;
```

(playground link)

Info

Raw pointers are unsafe to dereference and require `unsafe` blocks.

```
unsafe {
    println!("Value: {}", *r);
}
```

(playground link)

Is every pointer a reference?

5.4. Pointers

Every reference can be converted to a raw pointer, using a cast with `as`:

```rust
let x: i32 = 10;
let r: *const i32 = &x as *const i32;
```

(playground link)

Info

Raw pointers are unsafe to dereference and require `unsafe` blocks.

```rust
unsafe {
    println!("Value: {}", *r);
}
```

(playground link)

Is every pointer a reference?

No. References have extra compile-time checks to **prevent dangling** pointers.

Is every reference a pointer?

5.4. Pointers

Every reference can be converted to a raw pointer, using a cast with `as`:

```rust
let x: i32 = 10;
let r: *const i32 = &x as *const i32;
```
(playground link)

Info

Raw pointers are unsafe to dereference and require `unsafe` blocks.

```rust
unsafe {
    println!("Value: {}", *r);
}
```
(playground link)

Is every pointer a reference?

No. References have extra compile-time checks to **prevent dangling** pointers.

Is every reference a pointer?

Yes.

5.5. Exclusive references

Also called mutable references.

Info

Exclusive references (`&mut T`) allow mutation of the referenced object.

```rust
fn main() {
    let mut point = (1, 2);
    let x_coord = &mut point.0;
    // let y_coord = &mut point.1;
    *x_coord = 20;
    println!("point: {point:?}");
}
```

(playground link)

What happens if we uncomment?

5.5. Exclusive references

Also called mutable references.

Info

Exclusive references (`&mut T`) allow mutation of the referenced object.

```rust
fn main() {
    let mut point = (1, 2);
    let x_coord = &mut point.0;
    // let y_coord = &mut point.1;
    *x_coord = 20;
    println!("point: {point:?}");
}
```

(playground link)

What happens if we uncomment?

Illegal: cannot borrow `point` as mutable more than once at a time.

Warning

One exclusive reference can exist at a time!

5.6. Exclusive vs. shared

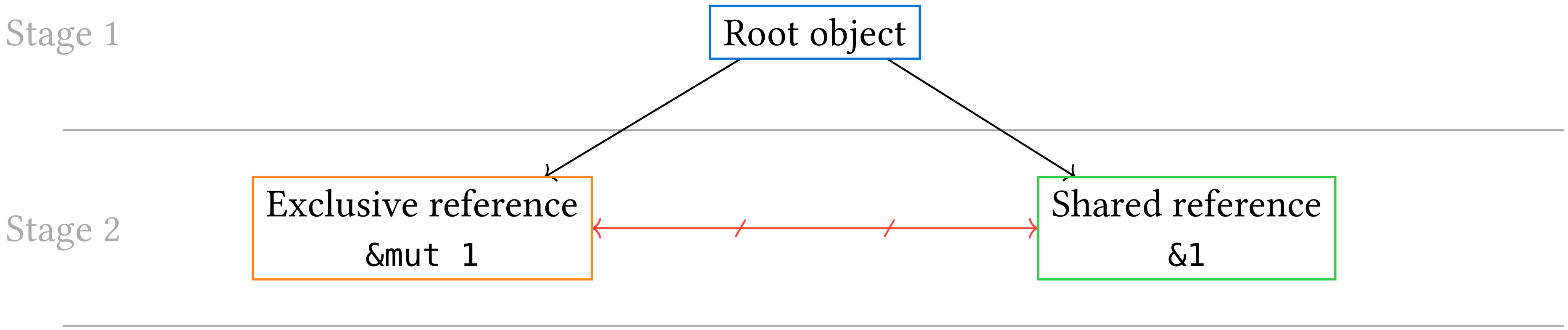
Exclusive references cannot coexist with shared references.

Stage 1

Root object

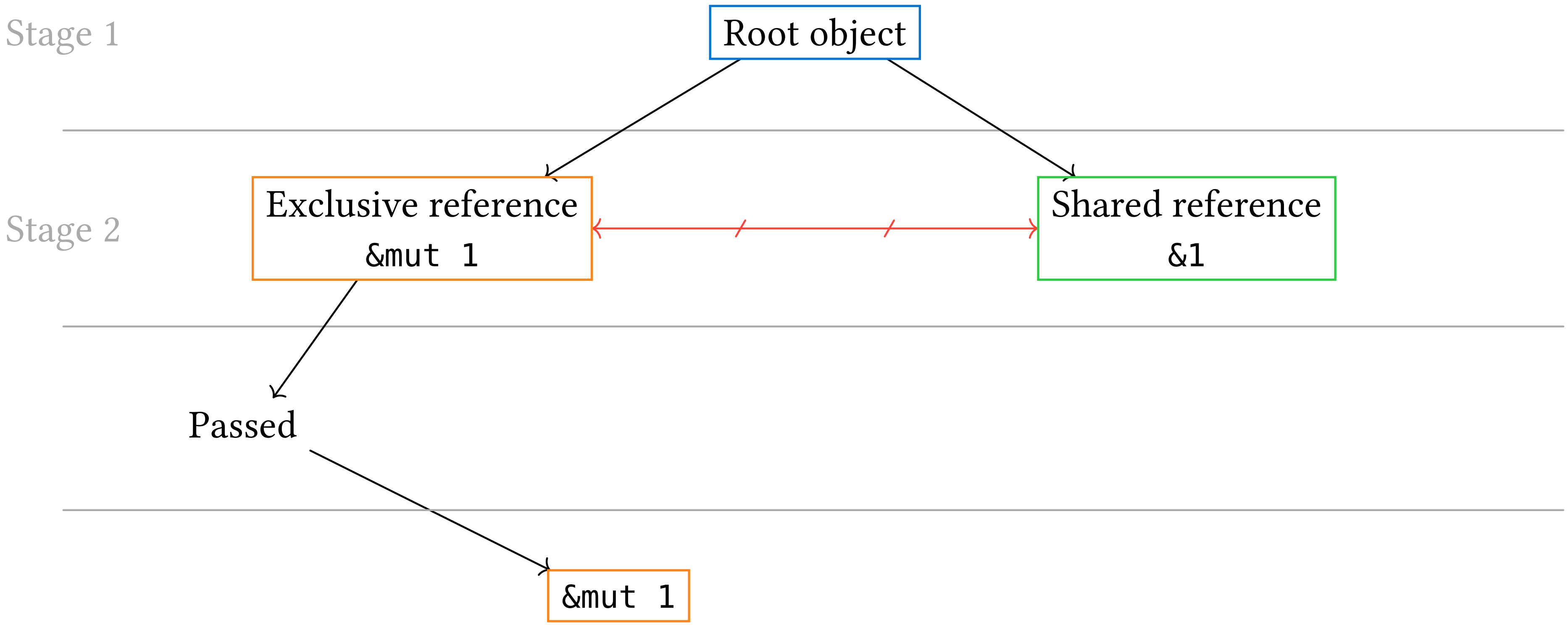
5.6. Exclusive vs. shared

Exclusive references cannot coexist with shared references.



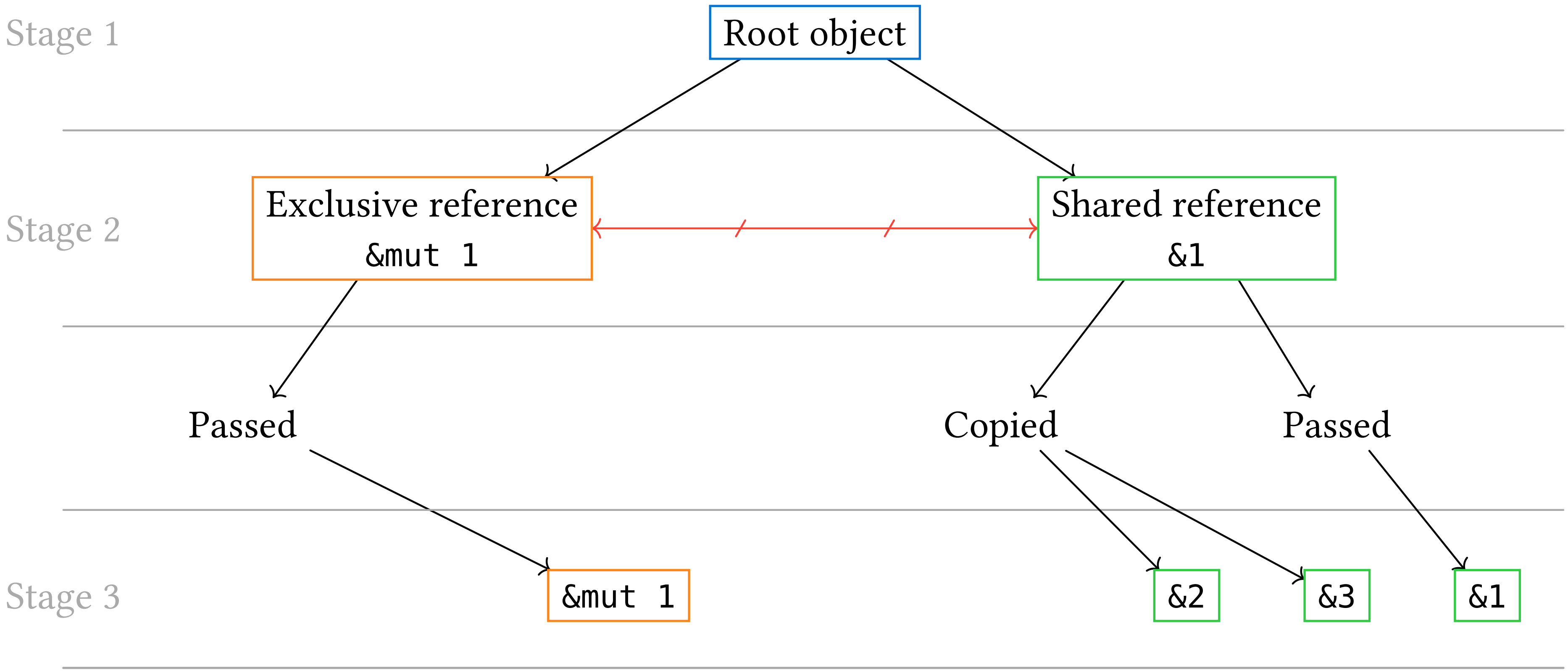
5.6. Exclusive vs. shared

Exclusive references cannot coexist with shared references.



5.6. Exclusive vs. shared

Exclusive references cannot coexist with shared references.



5.7. Kinds of `mut`

```rust
let mut x_coord: &i32;
let x_coord: &mut i32;
```

(playground link)

What is the difference between these statements?

5.7. Kinds of `mut`

```rust
let mut x_coord: &i32;
let x_coord: &mut i32;
```

(playground link)

What is the difference between these statements?

- `let mut x_coord: &i32;`: An immutable reference that can be reassigned to point to different `i32` values.
- `let x_coord: &mut i32;`: An exclusive reference that allows modifying the `i32` value it points to.

Info

Mutable references don't have to be assigned to `mut` variables to be able to mutate the referenced value.

5.7. Kinds of `mut`

There used to be discussion about having two different names for what is now both called `mut`:

5.7. Kinds of `mut`

There used to be discussion about having two different names for what is now both called `mut`:

Re-assignment `mut`

```rust
let mut a = 1;
a = 2; // You can re-assign the variable

impl {
  fn some_method(mut self) {
    // You can re-assign to self.
    self = some_thing(); // not very useful
    self.field = 2; // may be useful
  }
}
```

(playground link)

5.7. Kinds of `mut`

There used to be discussion about having two different names for what is now both called `mut`:

Re-assignment `mut`

```rust
let mut a = 1;
a = 2; // You can re-assign the variable

impl {
  fn some_method(mut self) {
    // You can re-assign to self.
    self = some_thing(); // not very useful
    self.field = 2; // may be useful
  }
}
```

(playground link)

Pointer `mut`

```rust
let mut a = 1;
let b = &mut a; // Need assignment `mut` on `a`
*b = 2; // You can assign through the pointer
```

(playground link)

Both are related to mutability, but are completely separate.

5.8. Slices

Info

A slice is a **view** into a contiguous sequence. (Like a horizontal continuous window)

```rust
fn main() {
    let a: [i32; 6] = [10, 20, 30, 40, 50, 60];
    println!("a: {a:?}");
    let s: &[i32] = &a[2..4];
    println!("s: {s:?}");
}
```

(playground link)

5.8. Slices

Info

A slice is a **view** into a contiguous sequence. (Like a horizontal continuous window)

```rust
fn main() {
    let a: [i32; 6] = [10, 20, 30, 40, 50, 60];
    println!("a: {a:?}");
    let s: &[i32] = &a[2..4];
    println!("s: {s:?}");
}
```

(playground link)

Can we drop the end index in the slice?

5.8. Slices

Info

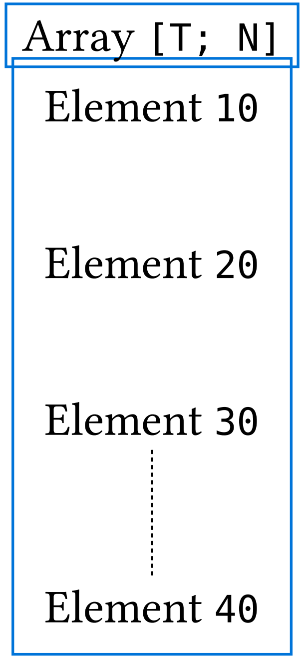
A slice is a **view** into a contiguous sequence. (Like a horizontal continuous window)

```
fn main() {
    let a: [i32; 6] = [10, 20, 30, 40, 50, 60];
    println!("a: {a:?}");
    let s: &[i32] = &a[2..4];
    println!("s: {s:?}");
}
```
(playground link)

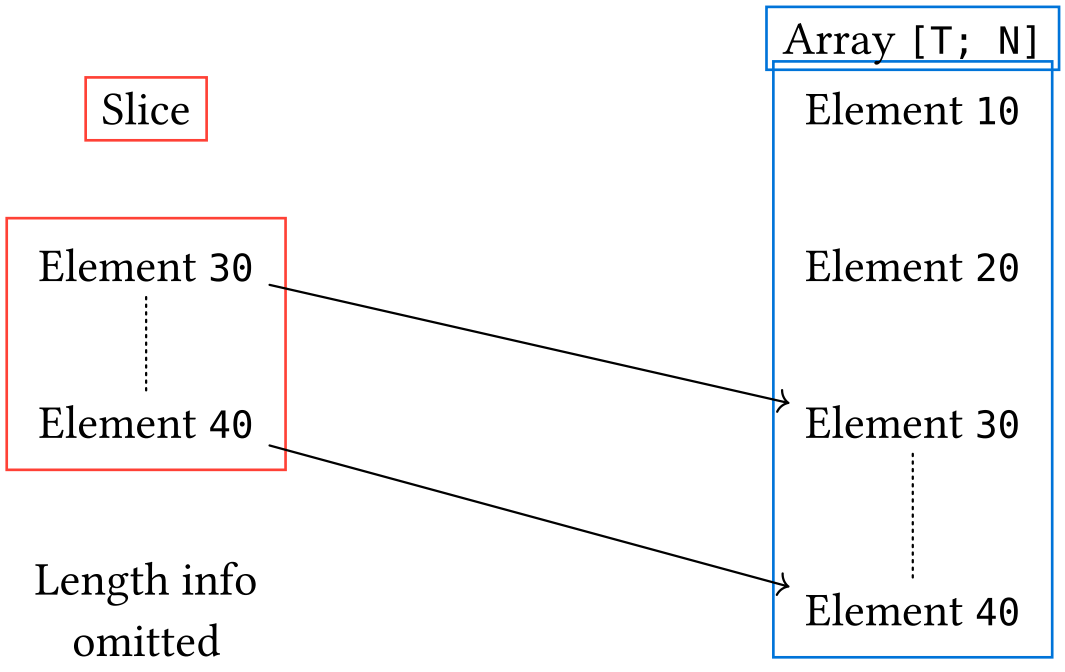
Can we drop the end index in the slice?

Yes, it defaults to the length of the array.

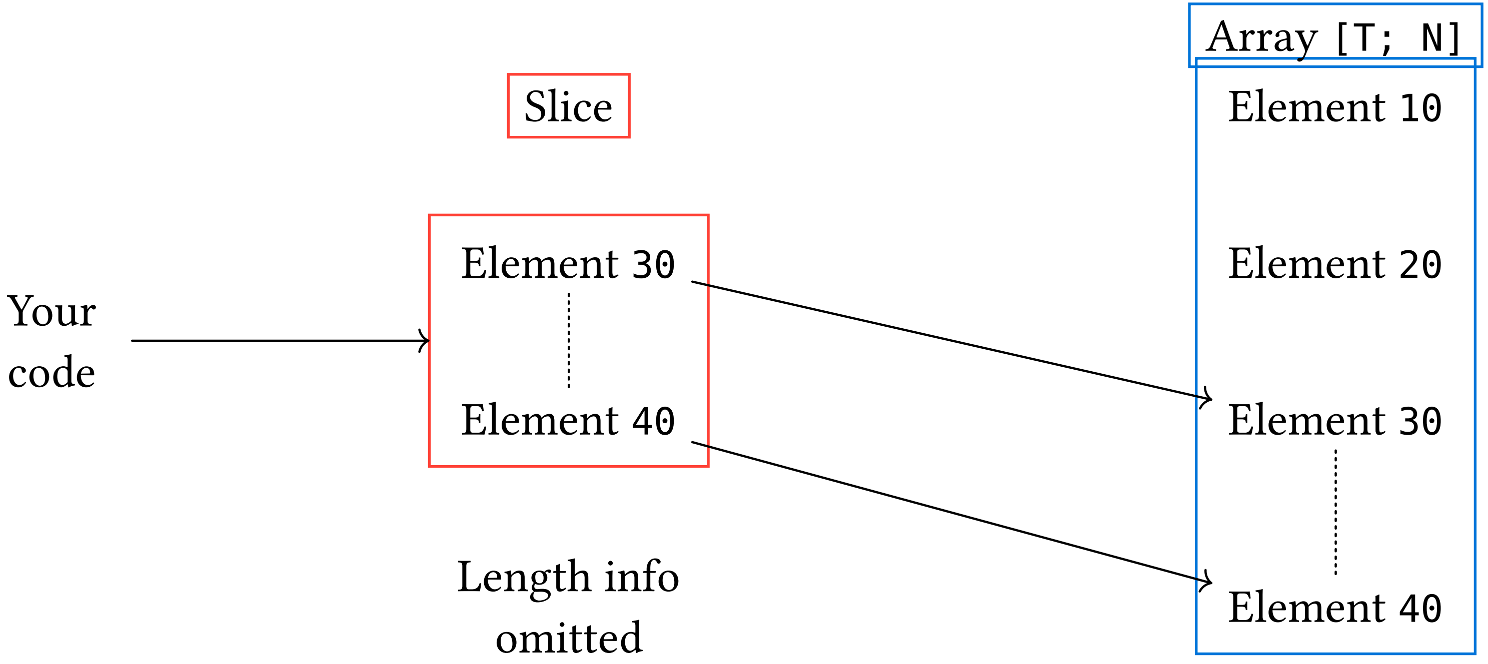
5.8. Slices



5.8. Slices

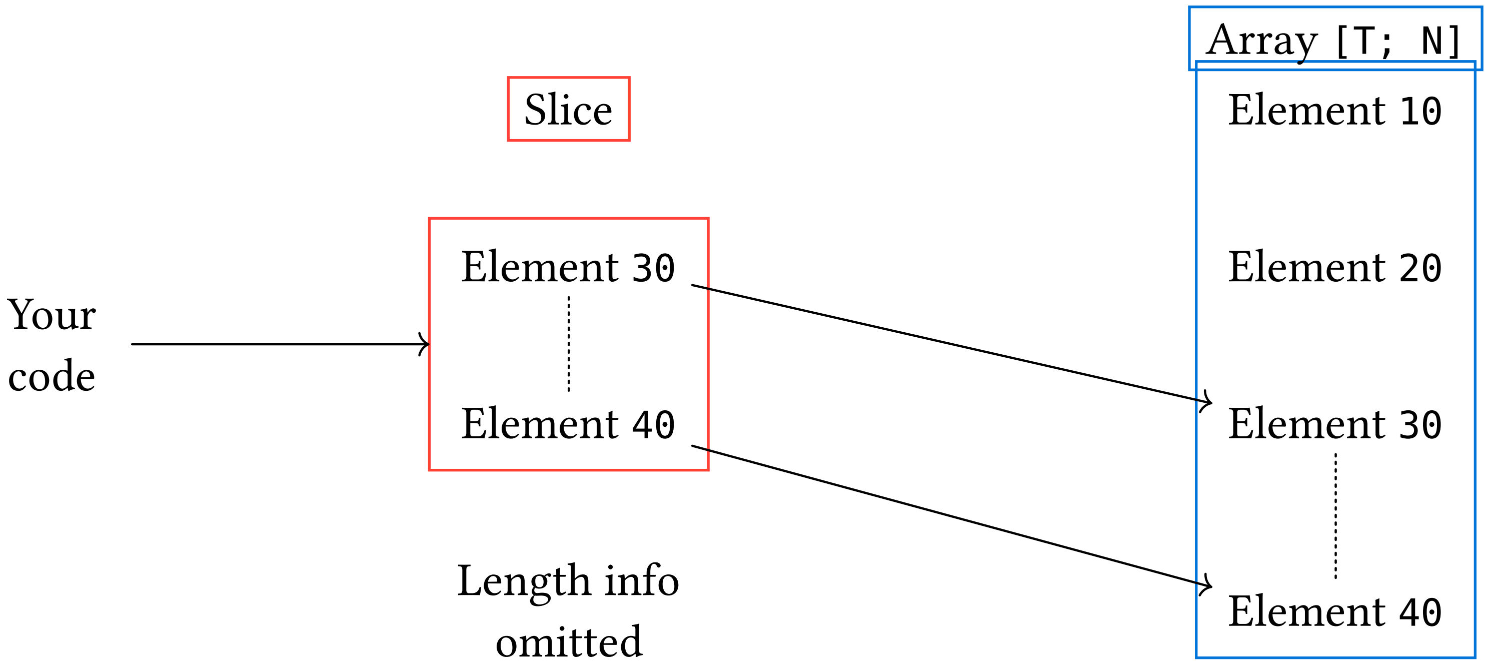

Slice
Element 30
Element 40
Length info omitted
Array [T; N]
Element 10
Element 20
Element 30
Element 40

5.8. Slices



What happens when you have the slice, but destroy the original array?

5.8. Slices



What happens when you have the slice, but destroy the original array?

Not possible in normal Rust. (See later about lifetimes)

5.8. Slices

More limitations of slices:

- You cannot append elements to a slice.
- You cannot grow a slice.
- You cannot shrink a slice.

However, slices have unique functionality defined.

Warning

Data carriers like arrays may refer you to the methods of the associated slices.

(see next slides about strings)

5.9. Strings


```rust
fn main() {
    let s1: &str = "World";
    println!("s1: {s1}");

    let mut s2: String = String::from("Hello ");
    println!("s2: {s2}");

    s2.push_str(s1);
    println!("s2: {s2}");

    let s3: &str = &s2[2..9];
    println!("s3: {s3}");
}
```

(playground link)

Two new types:

- A `&str` is an immutable slice of a `String`.
- A `String` is mutable vector of UTF-8 characters.

For the C++ programmers:

- `&str` is like `std::string_view`
- `String` is like `std::string`

Usage:

- `String::from(&str)` creates a `String` from a `&str`.
- `&String` coerces to `&str` automatically.
- String interpolation with formatting macros: `println!`, `format!`, `dbg!`, etc.

5.9. Strings

mutable `String`

Character `H`

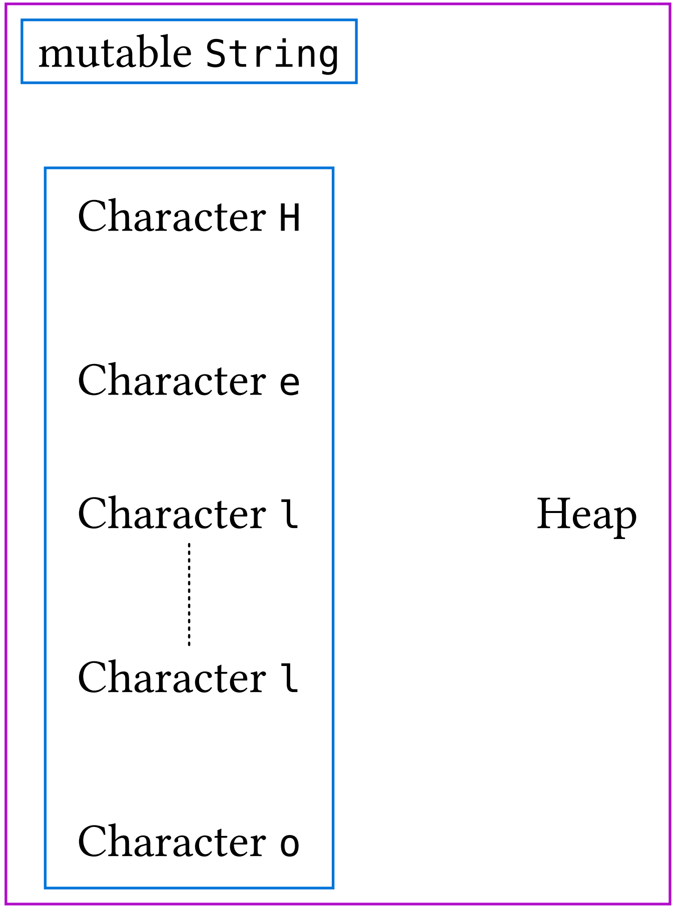
Character `e`

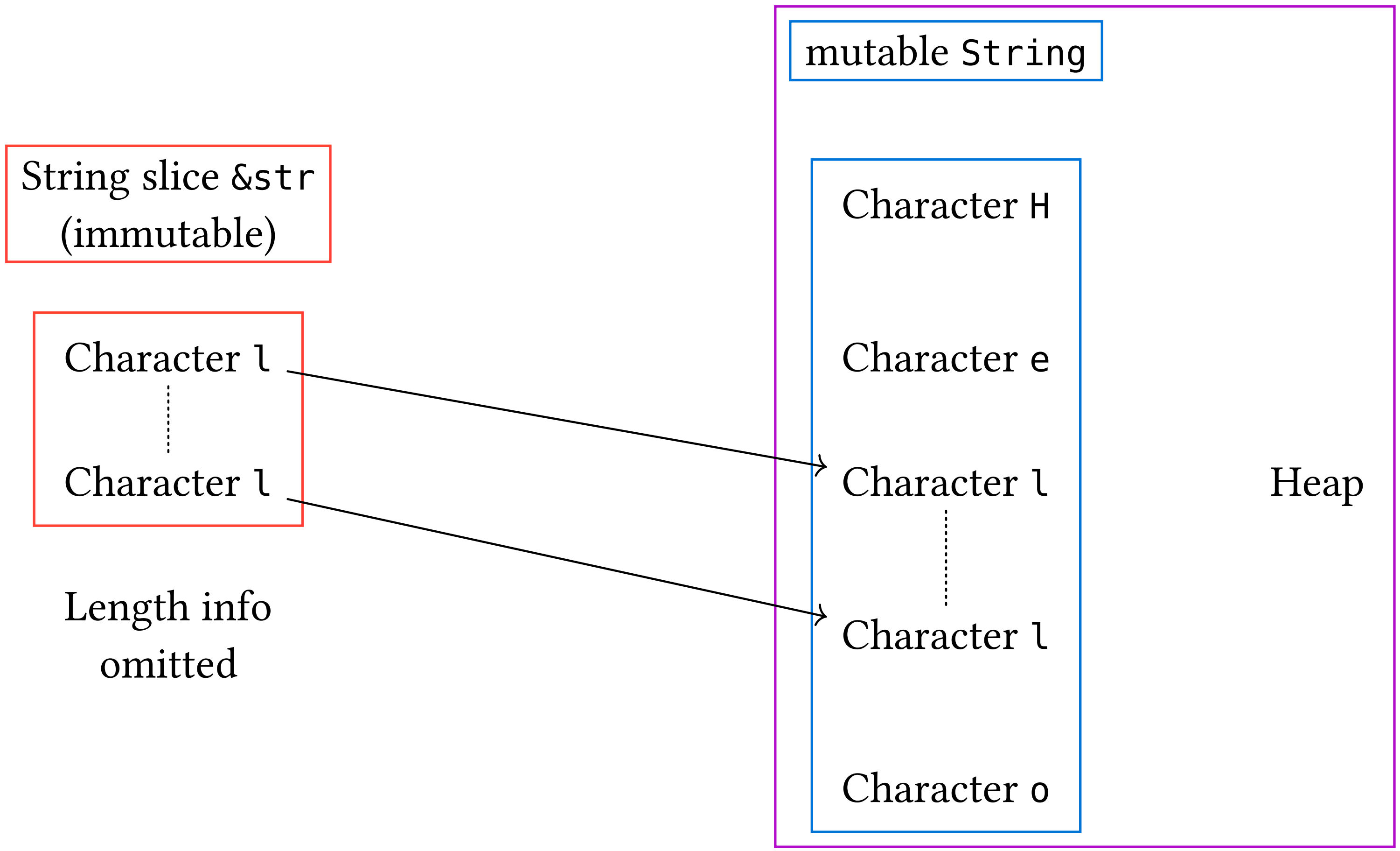
Character `l`

Character `l`

Character `o`

5.9. Strings


mutable String
Character H
Character e
Character l
Character l
Character o
Heap


mutable String
String slice &str
(immutable)
Character l
Character l
Length info
omitted
Character H
Character e
Character l
Character l
Character o
Heap

5.9. Strings

Is it possible to have a mutable string slice (`&mut str`)?

5.9. Strings

Is it possible to have a mutable string slice (`&mut str`)?

No.

5.9. Strings

Is it possible to have a mutable string slice (`&mut str`)?

No.

Why can't we have `&mut str`?

Is it possible to have a mutable string slice (`&mut str`)?

No.

Why can't we have `&mut str`?

Because strings are UTF-8 encoded and modifying individual bytes could create invalid UTF-8. Use `String` for mutation or `&mut [u8]` for byte manipulation.

5.9. Strings

Is it possible to have a mutable string slice (`&mut str`)?

No.

Why can't we have `&mut str`?

Because strings are UTF-8 encoded and modifying individual bytes could create invalid UTF-8. Use `String` for mutation or `&mut [u8]` for byte manipulation.

When should you use `String` vs `&str` in function parameters?

5.9. Strings

Is it possible to have a mutable string slice (`&mut str`)?

No.

Why can't we have `&mut str`?

Because strings are UTF-8 encoded and modifying individual bytes could create invalid UTF-8. Use `String` for mutation or `&mut [u8]` for byte manipulation.

When should you use `String` vs `&str` in function parameters?

Prefer `&str` for parameters (more flexible - accepts both `String` and `&str`). Use `String` when you need ownership or will modify it.

5.9. Strings

What if you **don’t want UTF-8** encoding?,

5.9. Strings

What if you **don't want UTF-8** encoding?,

Youuse byte slices of type `&[u8]`:

```
println!("{:?}", b"abc");
println!("{:?}", &[97, 98, 99]);
```

(playground link)

Are you going "inception", use '\' escapes (or a special Rust feature '#'):

```
println!(r#"<a href="link.html">link</a>"#);
println!("<a href=\"link.html\">link</a>");
```

(playground link)

5.10. Reference validity

```rust
fn main() {
    let x_ref = {
        let x = 10;
        &x
    };
    dbg!(x_ref);
}
```

(playground link)

5.10. Reference validity

```rust
fn main() {
    let x_ref = {
        let x = 10;
        &x
    };
    dbg!(x_ref);
}
```

(playground link)

References in Rust are always safe to use, why?

5.10. Reference validity

```rust
fn main() {
    let x_ref = {
        let x = 10;
        &x
    };
    dbg!(x_ref);
}
```

(playground link)

References in Rust are always safe to use, why?

- references can never be null
- references can't outlive the data they point to

(See next lectures)

5.11. Exercise: vectors

Write some utility functions for calculating the magnitude and normalizing 3D vectors.

Find the statement of the exercise here: `session1/examples/s1e4-vectors.rs`.

6.1. Named structs

Exactly the same as in C/C++:

```rust
struct Person {
    name: String,
    age: u8,
}

fn describe(person: &Person) {
    println!("{} is {} years old", person.name, person.age);
}
```

(playground link)

6.1. Named structs

6.1.1. Usage

```rust
struct Person {
    name: String,
    age: u8,
}

fn main() {
    let name = String::from("Peter");
    let mut peter = Person {
        name,
        age: 27,
    };
    describe(&peter);
    peter.age = 28;
}
```

(playground link)

6.1. Named structs

Warning

Struct fields do not support default values.

Use the `Default` trait to implement default values (see later).

6.1. Named `struct`s

6.1.2. Inheritance

How does Rust handle `struct` inheritance (like class inheritance in other languages?

6.1. Named structs

6.1.2. Inheritance

How does Rust handle `struct` inheritance (like class inheritance in other languages?

Rust explicitly forbids `struct` inheritance.

Warning

Inheritance in Rust is replaced by trait composition.

Info

Only abstract interfaces can be inherited, concrete types such as `struct`s cannot.

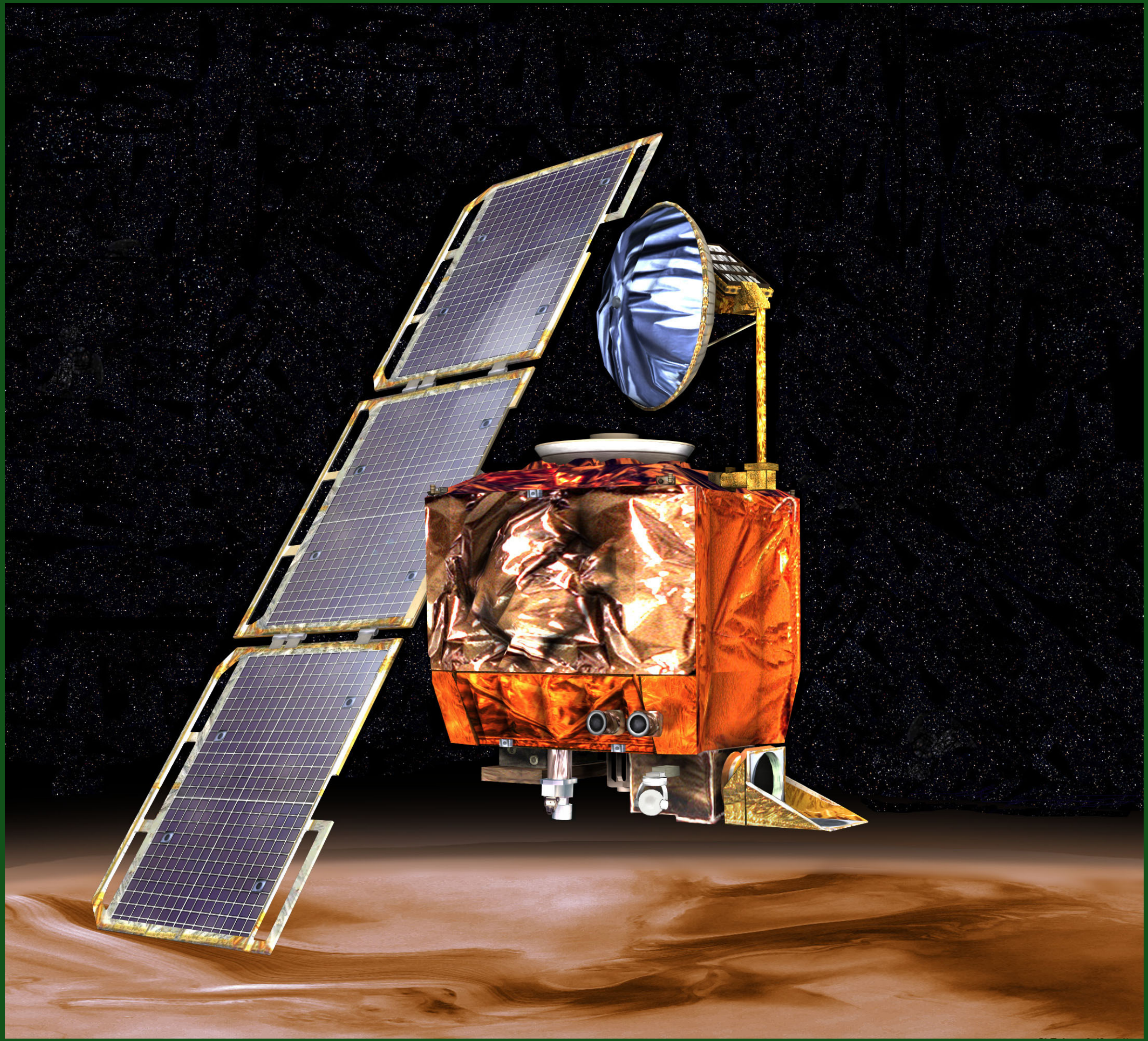
6.2. Tuple structs

If the field names are unimportant, you can use a tuple struct:

```rust
  struct Point(i32, i32);

fn main() {
    let p = Point(17, 23);
    println!("({}, {})", p.0, p.1);
}
```

(playground link)



6.2. Tuple `structs`

What is the name of this satellite?

6.2. Tuple `structs`

What is the name of this satellite?

Mars Climate Orbiter

Why did it fail?

6.2. Tuple structs

What is the name of this satellite?

Mars Climate Orbiter

Why did it fail?

A unit conversion error: one team used imperial units (pounds of force), the other metric (Newtons).

6.2. Tuple `structs`

6.2.1. Usage

This is often used for single-field wrappers (called newtypes):

6.2. Tuple structs

```rust
struct PoundsOfForce(f64);
struct Newtons(f64);

fn compute_thruster_force() -> PoundsOfForce {
    todo!("Ask a rocket scientist at NASA")
}

fn set_thruster_force(force: Newtons) {
    // ...
}

fn main() {
    let force = compute_thruster_force();
    set_thruster_force(force);
}
```

(playground link)

6.2. Tuple `structs`

6.2.2. Remarks

- Great way to encode additional information about the value in a primitive type
- Value passed some validation when it was created, so you no longer have to validate it again at every use

How do you prevent invalidation of the value after validation during creation?

6.2. Tuple `structs`

6.2.2. Remarks

- Great way to encode additional information about the value in a primitive type
- Value passed some validation when it was created, so you no longer have to validate it again at every use

How do you prevent invalidation of the value after validation during creation?

Keep the inner field private by adding `pub` only to the struct, not to its fields.

6.3. Enums

The enum keyword allows the creation of a type which has a few different variants:

```rust
#[derive(Debug)]
enum Direction {
    Left,
    Right,
}
```

(playground link)

```rust
#[derive(Debug)]
enum PlayerMove {
    Pass,
    Run(Direction),
    Teleport { x: u32, y: u32 },
}
```

(playground link)

```rust
fn main() {
    let dir = Direction::Left;
    let player_move: PlayerMove = PlayerMove::Run(dir);
    println!("On this turn: {player_move:?}");
 }
```

(playground link)

6.3. Enums

6.3.1. Storage of `enum`

Info

An `enum` occupies enough space to store **its largest variant** plus some extra space to store which variant it currently is.

```rust
  #[repr(u32)]
enum Bar {
    A, // 0
    B = 10000,
    C, // 10001
}

fn main() {
    println!("A: {}", Bar::A as u32);
    println!("B: {}", Bar::B as u32);
    println!("C: {}", Bar::C as u32);
}
```

(playground link)

6.3. Enums

Without repr, the discriminant type takes 2 bytes, because 10001 fits 2 bytes.

6.3. Enums

Who is Tony Hoare and what was his billion dollar mistake?

6.3. Enums

Who is Tony Hoare and what was his billion dollar mistake?

Tony Hoare is a British computer scientist who invented the null reference in 1965 while designing the ALGOL W programming language.

6.3.2. Option enum

The simplest example of an enum is the `Option<T>` type, which represents either a value of type `T` or no value at all:

```
enum Option<T> {
    None,
    Some(T),
}
```

(playground link)

How much space does `Option<&u8>` occupy?

6.3.2. Option enum

The simplest example of an enum is the `Option<T>` type, which represents either a value of type `T` or no value at all:

```
enum Option<T> {
    None,
    Some(T),
}
```

(playground link)

How much space does `Option<&u8>` occupy?

The same as a single reference (8 bytes on 64-bit), because `None` is represented as a null pointer, so no extra discriminant space is needed.

6.3. Enums

6.3.2. Option enum

The simplest example of an enum is the `Option<T>` type, which represents either a value of type `T` or no value at all:

```
enum Option<T> {
    None,
    Some(T),
}
```

(playground link)

How much space does `Option<&u8>` occupy?

The same as a single reference (8 bytes on 64-bit), because `None` is represented as a null pointer, so no extra discriminant space is needed.

Info

For the functional programmers: `Option` is also a **monad**, but in Rust monadic `do` notation is replaced by the short-circuiting `?` operator.

6.4. Compile-time const


```rust
const fn is_prime(n: u32) -> bool {
    if n < 2 { return false; }
    let mut i = 2;
    while i * i <= n {
        if n % i == 0 { return false; }
        i += 1;
    }
    true
}

fn main() {
    println!("First 25 primes: {:?}",
             PRIMES_BELOW_100);
}
```

(playground link)

```rust
const PRIMES_BELOW_100: [u32; 25] = {
    let mut primes = [0; 25];
    let mut count = 0;
    let mut n = 2;
    while n < 100 {
        if is_prime(n) {
            primes[count] = n;
            count += 1;
        }
        n += 1;
    }
    primes
};
```

(playground link)

Info

The prime sieve runs at **compile time**, resulting in zero runtime overhead!

6.4. Compile-time `const`

Info

Similar to C++'s `constexpr`, Zig's `comptime`

6.4. Compile-time `const`

Info

Similar to C++'s `constexpr`, Zig's `comptime`

What are the limitations of `const fn`?

6.4. Compile-time `const`

Info

Similar to C++'s `constexpr`, Zig's `comptime`

What are the limitations of `const fn`?

- Cannot allocate heap memory
- Cannot call non-`const` functions
- Cannot use floating-point operations (being relaxed in newer Rust versions)
- Limited to a subset of Rust operations

6.4. Compile-time `const`

Info

Similar to C++'s `constexpr`, Zig's `comptime`

What are the limitations of `const fn`?

- Cannot allocate heap memory
- Cannot call non-`const` functions
- Cannot use floating-point operations (being relaxed in newer Rust versions)
- Limited to a subset of Rust operations

Why these restrictions?

6.4. Compile-time `const`

Info

Similar to C++'s `constexpr`, Zig's `comptime`

What are the limitations of `const fn`?

- Cannot allocate heap memory
- Cannot call non-`const` functions
- Cannot use floating-point operations (being relaxed in newer Rust versions)
- Limited to a subset of Rust operations

Why these restrictions?

To guarantee deterministic, side-effect-free compile-time evaluation with predictable performance.

6.5. Static

Static variables will live during the whole execution of the program, and therefore will not move:

```rust
static BANNER: &str = "Welcome to RustOS 3.14";

fn main() {
    println!("{BANNER}");
}
```

(playground link)

Info

Accessible from any thread.

Warning

Not inlined upon use and have an actual associated memory location.

Why is this often used in embedded Rust?

6.5. Static

Static variables will live during the whole execution of the program, and therefore will not move:

```rust
static BANNER: &str = "Welcome to RustOS 3.14";

fn main() {
    println!("{BANNER}");
}
```

(playground link)

Info

Accessible from any thread.

Warning

Not inlined upon use and have an actual associated memory location.

Why is this often used in embedded Rust?

Stable addresses are often required. Memory shared between cores. No heap allocation is available.

6.5.1. `static OnceLock` pattern

When a `static` needs to be initialised at runtime (just once) and is read-only, use `OnceLock`:

```
use std::sync::OnceLock;

static CONFIG: OnceLock<Config> = OnceLock::new();

fn main() {
    CONFIG.get_or_init(|| {
        Config::load("config.toml")
    });
}
```

(playground link)

6.5.2. Plant pot workshop

A few months ago, at this location, we built an embedded Rust plant pot monitor together

Instructions available at https://github.com/sysghent/plant-pot.

```
static HUMIDITY_PUBSUB_CHANNEL:
  PubSubChannel<CriticalSectionRawMutex, f32, 1, 3, 1> =
    PubSubChannel::new();

#[main]
async fn main(spawner: Spawner) -> ! {
    let p = embassy_rp::init(config::Config::default());
    let adc_component = Adc::new(p.ADC, Irqs, Config::default());
    // ...
}
```

(playground link)

6.5.2. Plant pot workshop

A few months ago, at this location, we built an embedded Rust plant pot monitor together

Instructions available at https://github.com/sysghent/plant-pot.

```
static HUMIDITY_PUBSUB_CHANNEL:
  PubSubChannel<CriticalSectionRawMutex, f32, 1, 3, 1> =
    PubSubChannel::new();

#[main]
async fn main(spawner: Spawner) -> ! {
    let p = embassy_rp::init(config::Config::default());
    let adc_component = Adc::new(p.ADC, Irqs, Config::default());
    // ...
  }
```

(playground link)

In embedded Rust, you either use the `OnceLock` or `const` functions to initialise `static` variables.

Where is the `const fn` function in this fragment and how to find out?

6.5.2. Plant pot workshop

A few months ago, at this location, we built an embedded Rust plant pot monitor together

Instructions available at https://github.com/sysghent/plant-pot.

```rust
static HUMIDITY_PUBSUB_CHANNEL:
  PubSubChannel<CriticalSectionRawMutex, f32, 1, 3, 1> =
    PubSubChannel::new();

#[main]
async fn main(spawner: Spawner) -> ! {
    let p = embassy_rp::init(config::Config::default());
    let adc_component = Adc::new(p.ADC, Irqs, Config::default());
    // ...
  }
```

(playground link)

In embedded Rust, you either use the `OnceLock` or `const` functions to initialise `static` variables.

Where is the `const fn` function in this fragment and how to find out?

(In this example `PubSubChannel::new()` is a `const fn`.)

6.6. Summary: World map

Compile-time world

6.6. Summary: World map

Compile-time world

`const` variables

6.6. Summary: World map

Compile-time world

`const` variables

`const fn` functions

6.6. Summary: World map

Compile-time world

`const` variables

`const fn` functions

`const` generics

6.6. Summary: World map

Compile-time world
`const` variables
`const fn` functions
`const` generics

6.6. Summary: World map

Compile-time world	Runtime world
`const` variables	
`const fn` functions	
`const` generics	

6.6. Summary: World map

Compile-time world	Wild West: embedded	Runtime world
`const` variables		
`const fn` functions		
`const` generics		

6.6. Summary: World map

Compile-time world	Wild West: embedded	Runtime world
`const` variables	`OnceLock` `LazyLock` "init on use"	
`const fn` functions		
`const` generics		

6.6. Summary: World map

Compile-time world	Wild West: embedded	Runtime world
`const` variables	`OnceLock` `LazyLock` “init on use”	
`const fn` functions	`unsafe` code	
`const` generics		

6.6. Summary: World map

Compile-time world	Runtime world: Wild West: embedded	Runtime world: Safe Rust

`const` variables	`OnceLock`
	`LazyLock`
	“init on use”	

`const fn` functions	`unsafe` code	
`const` generics		

6.6. Summary: World map

Compile-time world	**Runtime world**	
	Wild West: embedded	Safe Rust

`const` variables	`OnceLock`
	`LazyLock`
	“init on use”
		Heap allocation

`const fn` functions	`unsafe` code	
`const` generics		

6.6. Summary: World map

Compile-time world	**Runtime world** Wild West: embedded	Safe Rust
`const` variables	`OnceLock` `LazyLock` “init on use”	Heap allocation
`const fn` functions	`unsafe` code	Normal functions
`const` generics		

6.6. Summary: World map

Compile-time world	**Runtime world**		
	Wild West: embedded	Safe Rust	

`const` variables	`OnceLock`
	`LazyLock`
	“init on use”
		Heap allocation	Mutable variables

`const fn` functions	`unsafe` code	Normal functions	
`const` generics			

6.7. Exercise: Elevator Events

6.7.1. Goal

Create a data structure to represent an event in an elevator control system.

Complete the code in `session1/examples/s1e3-elevator.rs`.